

Universidad de Costa Rica

Pruebas de software

Informe: Etapa 1

Docente:

Minor Sandí Salazar

Integrantes:

Angel Mena Coudin | C34789

Gabriel Serrano Rojas | C17497

Juan José Víquez Ríos | C38567

Josué Torres Sibaja | C37853

Rolando Villavicencio Gonzales | C28489

Año:

2026

Introducción

En el siguiente documento se presenta la primera etapa del proyecto de pruebas de software, cuyo propósito es establecer las bases para evaluar la calidad de una solución que integra modelos de aprendizaje automático, servicios de API, una interfaz de usuario y persistencia de datos.

La solución contempla tres modelos: riesgo de diabetes, monto de préstamo y riesgo académico.

Durante esta etapa se documentan la estructura inicial del repositorio, el plan de trabajo, el alcance inicial de las pruebas y el diseño de los casos iniciales de pruebas.

Además, se describe la instalación y verificación del ambiente de pruebas, incluyendo Python, las dependencias administradas mediante `uv`, la ejecución de pruebas con `pytest` mediante `uv`, el análisis estático con `Pylint` mediante `uv` y la conexión con Supabase.

Objetivos

El objetivo del proyecto es someter la aplicación a un proceso de pruebas, aplicando de forma práctica los conocimientos, estándares, modelos y herramientas de las pruebas de software. Esto incluye principios de pruebas, gestión del proceso de pruebas, verificación y validación, entre otros conceptos clave para evaluar y garantizar la calidad del sistema.

Descripción

Problema a abordar:

Aplicación de pruebas de software para asegurar la calidad sobre una solución que integra modelos de aprendizaje automático.

Funcionalidades a probar:

- Selección del modelo y despliegue del formulario correspondiente a sus parámetros.
- Captura de los parámetros de entrada de cada modelo.
- Validación de las entradas escritas en la página web, antes de enviar la solicitud a la API.
- Presentación del resultado.
- Manejo y presentación de los errores devueltos por la API.
- Validación del contrato de entrada por modelo, independiente de la validación hecha en la interfaz.
- Invocación del modelo correspondiente según la ruta/endpoint solicitado.
- Devolución de la respuesta con la estructura del contrato: fecha, hora, modelo, predicción, métrica e id_ejecucion.
- Manejo de errores con los códigos HTTP correspondientes.
- Seguridad básica de las rutas expuestas: entradas maliciosas y exposición de información sensible.
- Cálculo de la predicción para cada uno de los tres modelos.
- Manejo de valores extremos o inesperados.
- Precisión numérica del resultado, incluyendo el redondeo (en particular para M08, que devuelve un monto).
- Almacenamiento de la ejecución y su resultado.
- Consulta del historial de ejecuciones almacenado.
- Comportamiento del sistema cuando la base de datos no está disponible.
- Pruebas de carga o rendimiento bajo volumen alto de solicitudes.

- Autenticación y autorización de usuarios.
- Despliegue en un ambiente de producción.
- Automatización de pruebas de UI end-to-end (por ahora se contemplan pruebas manuales de usabilidad).
- El desarrollo se encuentra en fase inicial: solo el esqueleto del repositorio y el ambiente están configurados. La API, el UI y los tres modelos aún no están implementados, por lo que este diseño es preliminar y se refinará en las siguientes etapas.

Alcance inicial:

En esta etapa se documenta la estructura inicial del repositorio, el plan de trabajo, el plan inicial de pruebas y el diseño de los casos iniciales de pruebas. Además, se describe la instalación y verificación del ambiente de pruebas, la ejecución de pruebas, el análisis estático, todo lo anterior mediante la herramienta **uv**, y finalmente la conexión con Supabase.

Repositorio Git

Dirección del repositorio

URL:

[Elance al repositorio](#)

Explicación:

URL a un repositorio remoto en github.

Estructura

Diagrama:

```
proyecto-pruebas/  
├── docs/  
│   └── ...  
├── evidencias/  
│   ├── diseño_de_pruebas/  
│   ├── instalacion/  
│   ├── plan_de_pruebas/  
│   ├── repositorio/  
│   └── verificacion/  
├── src/  
│   └── proyecto_pruebas/  
├── tests/  
└── README.md
```

Explicación:

La estructura actual está basada en la recomendación del profesor. Con ligeras alteraciones al no hacer uso de requirements.txt, ya que vamos a utilizar la herramienta uv para gestionar los paquetes y el proyecto.

Por otro lado, la carpeta de evidencia se descompone en carpetas específicas para cada evidencia solicitada.

Archivos iniciales

Captura:

docs	Añadir estructura inicial de direc...
evidencias	Agregar estructura para la explic...
src	Verificar librerías con print().
tests	Verificar pytest.
.env.example	Añadir la plantilla del archivo .e...
.gitignore	Agregar .gitignore.
.python-version	Configurar el proyecto para el u...
README.md	Actualizar README con instrucc...
pyproject.toml	Verificar pytest.
uv.lock	Configurar el proyecto para el u...

Explicación:

Los archivos iniciales son específicamente archivos git y archivos relacionados a UV, que se generan automáticamente, necesarios para que el ambiente funcione.

Commits realizados

Se pueden visualizar en el repositorio, por medio del siguiente enlace directamente:

[Ver commits y estadísticas](#)

Plan de Trabajo

El plan de trabajo se armó en un google sheets que se puede visualizar desde el siguiente enlace:

[Ver plan de trabajo](#)

No es la versión final, se va ir actualizando conforme se vaya definiendo las cargas de trabajo del proyecto.

Plan Inicial de Pruebas**1. Alcance**

Qué se probará. La solución mínima construida por el equipo, que integra los tres modelos asignados:

- **M07 — Riesgo de diabetes** (Salud, clasificación binaria)
- **M08 — Monto de préstamo** (Finanzas, regresión)
- **M09 — Riesgo académico** (Educación, clasificación multiclase)

expuestos mediante una API que valida el contrato y orquesta la predicción, una interfaz de usuario que captura los parámetros y presenta el resultado, y persistencia de solicitudes/resultados en Supabase/PostgreSQL.

Funcionalidades objeto de prueba

Interfaz

- Selección del modelo y despliegue del formulario correspondiente a sus parámetros.
- Captura de los parámetros de entrada de cada modelo.
- Validación de las entradas escritas en la página web, antes de enviar la solicitud a la API.
- Presentación del resultado.
- Manejo y presentación de los errores devueltos por la API.

API

- Validación del contrato de entrada por modelo, independiente de la validación hecha en la interfaz.
- Invocación del modelo correspondiente según la ruta/endpoint solicitado.
- Devolución de la respuesta con la estructura del contrato: fecha, hora, modelo, predicción, métrica e id_ejecucion.
- Manejo de errores con los códigos HTTP correspondientes.
- Seguridad básica de las rutas expuestas: entradas maliciosas y exposición de información sensible.

Modelos

- Cálculo de la predicción para cada uno de los tres modelos.
- Manejo de valores extremos o inesperados.
- Precisión numérica del resultado, incluyendo el redondeo (en particular para M08, que devuelve un monto).

Base de datos

- Almacenamiento de la ejecución y su resultado.
- Consulta del historial de ejecuciones almacenado.
- Comportamiento del sistema cuando la base de datos no está disponible.

Fuera de alcance

- Pruebas de carga o rendimiento bajo volumen alto de solicitudes.
- Autenticación y autorización de usuarios.
- Despliegue en un ambiente de producción.
- Automatización de pruebas de UI end-to-end (por ahora se contemplan pruebas manuales de usabilidad).

Limitaciones conocidas

- El desarrollo se encuentra en fase inicial: solo el esqueleto del repositorio y el ambiente están configurados. La API, el UI y los tres modelos aún no están implementados, por lo que este diseño es preliminar y se refinará en las siguientes etapas.

2. Riesgos

Riesgo	Probabilidad	Impacto	Mitigación
Supabase no disponible o mal configurado	Media	Alto	Verificar la conexión al inicio de cada sesión de trabajo; documentar el procedimiento de configuración
Cambios en el contrato de la API durante el desarrollo	Alta	Alto	Mantener el contrato documentado y bajo control de versiones; automatizar pruebas de contrato
Inconsistencia entre los tres tipos de modelo (binario, regresión, multiclase) en la respuesta de la API	Media	Medio	Definir un contrato de respuesta común y validar cada tipo por separado
Disponibilidad desigual de los integrantes del equipo	Media	Medio	Repartir tareas pequeñas y verificables (roles pendientes de asignar)
Tiempo insuficiente antes de la entrega final (20 de noviembre)	Media	Alto	Priorizar la automatización mínima exigida y avanzar de forma incremental por etapa

3. Estrategia

La estrategia recorre los niveles de la **pirámide de pruebas**:

unitarias → integración → funcionales/sistema → aceptación, combinando **Verificación** (revisiones, análisis estático, pruebas unitarias) y **Validación** (pruebas funcionales y de aceptación) en cada fase.

Se aplicarán pruebas funcionales positivas y negativas en partes iguales, siguiendo la **regla del 50/50**:

la mitad de los casos valida que el sistema haga lo que debe, la otra mitad que no haga lo que no debe. Se complementan con pruebas de integración (UI→API, API→modelo, API→Supabase y el flujo completo) y no funcionales básicas (usabilidad, confiabilidad, seguridad).

El laboratorio de ML visto en clase añade dos categorías propias de sistemas de ML:

pruebas de validez de datos (que la entrada cumpla formato, tipo y rango esperado, tanto antes del preprocesamiento como en el contrato de la API) y **pruebas de calidad del modelo** (que las métricas de cada modelo, sección 7, no se degraden por debajo de un umbral mínimo aceptable).

Manuales y automatizadas se complementan, no compiten. Se automatizarán con pytest las pruebas unitarias de cada modelo, las pruebas de contrato de la API (positivas y negativas), la integración API→modelo, la persistencia API→Supabase y al menos un caso de borde, cumpliendo el mínimo de automatización exigido por el proyecto. Se ejecutarán de forma manual las pruebas de usabilidad de la interfaz, la exploración del flujo completo UI→API→modelo→Supabase y las pruebas no funcionales de confiabilidad ante indisponibilidad y seguridad básica.

Como **prueba estática** se incluye análisis estático automatizado (Pylint sobre `src/`). Cada caso de prueba del Diseño de Pruebas deberá declarar un **oráculo explícito**, sin el cual no se puede concluir si la prueba pasó o falló.

Técnicas de diseño aplicadas (Caja Negra, mínimo 3)

1. **Partición de equivalencia:** agrupa valores válidos e inválidos de los parámetros de entrada de cada modelo (ej. edad, ingresos, notas).
2. **Análisis de valores límite:** prueba los límites de rangos numéricos relevantes (edad mínima/máxima, montos de préstamo, notas límite del riesgo académico).
3. **Pruebas basadas en riesgo:** prioriza, entre los tres modelos, los flujos con mayor probabilidad o impacto de fallo.

4. Criterios

Criterios de entrada

- Contrato de API definido y documentado para los tres modelos.
- Ambiente de desarrollo configurado y verificado (Python, dependencias, Supabase).
- Modelos accesibles (entrenados o con lógica mínima de predicción).
- Plan de Pruebas y Diseño de Pruebas completos, con los datos de prueba definidos y los casos priorizados (cada uno con su resultado esperado).

Criterios de salida

- Casos de prueba ejecutados y resultados registrados para los tres modelos.
- Defectos registrados en GitHub Issues (severidad, prioridad, pasos, evidencia).
- Retests completados y regresión aplicada donde corresponda.
- Métricas calculadas y trazabilidad actualizada.
- Recomendación Go/No-Go sustentada (a emitir en una etapa posterior).

5. Recursos

Humanos

Nombre	% de participación	Asignación
Angel Mena Coudin	100%	Pendiente
Gabriel Serrano Rojas	100%	Pendiente
Juan José Víquez Ríos	100%	Pendiente
Josué Torres Sibaja	100%	Pendiente
Rolando Villavicencio González	100%	Pendiente

Los integrantes participarán de forma transversal en todas las actividades del proyecto (análisis, desarrollo, pruebas, documentación), sin roles fijos por componente.

- **Hardware:** equipo de cómputo personal de cada integrante.
- **Software y versión de Python:** Python 3.13+, gestionado con **uv**.
- **Librerías:** scipy, scikit-learn, pytest, pylint (ya instaladas); se agregarán las del framework de API, de UI y el cliente de Supabase.
- **Herramientas:** Git/GitHub, GitHub Issues (registro de defectos), Pylint (análisis estático).
- **Datos de prueba:** datasets por modelo (M07, M08, M09), proporcionados por el docente.
- **Ambiente de desarrollo:** entorno virtual gestionado por **uv** (**.venv**), variables de entorno vía **.env**.

- **Infraestructura adicional:** proyecto de Supabase (PostgreSQL en la nube).

6. Cronograma

Pendiente, depende de las fechas definidas en el plan de trabajo.

7. Métricas

Los defectos se clasificarán por **severidad** (impacto técnico) y **prioridad** (urgencia de corrección) de forma independiente.

Métricas propias de cada modelo

Propósito: confirmar que la predicción de cada modelo es confiable antes de sustentar su Go/No-Go:

- M07 (riesgo de diabetes, clasificación binaria): accuracy, precision, recall, F1.
- M08 (monto de préstamo, regresión): MAE, MSE/RMSE, R^2 .
- M09 (riesgo académico, clasificación multiclase): accuracy, precision, recall, F1, matriz de confusión.

Métricas del proceso de pruebas

Propósito: medir el avance y la calidad del proceso de pruebas para monitorear su progreso:

- Porcentaje de casos ejecutados, aprobados y fallidos.
- Cobertura funcional y cobertura negativa.
- Cantidad de defectos por severidad y por componente.
- Porcentaje de pruebas automatizadas.

Diseño inicial de Pruebas

Pendiente

Instalación del ambiente de Pruebas

Instalación de Python

Captura:

```
PS C:\WINDOWS\system32> powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
downloading uv 0.12.15 (x86_64-pc-windows-msvc)
installing to C:\Users\vique\.local\bin
uv.exe
uvx.exe
uvw.exe
everything's installed!
PS C:\WINDOWS\system32> uv --version
uv 0.12.15 (d35f1f270 2026-09-15 x86_64-pc-windows-msvc)
```

Explicación:

No se instala python directamente, sino que se instala uv, el mismo se encarga de instalar la versión de python que se necesita para el proyecto y las dependencias necesarias para el proyecto.

Creación del proyecto en UV (Creación del entorno virtual, instalación de dependencias y archivo de requirements.txt)

Captura:

```

PS C:\Users\vique\source\repos\proyecto-pruebas> uv --version
uv 0.12.15 (d35f1f270 2026-09-15 x86_64-pc-windows-msvc)
PS C:\Users\vique\source\repos\proyecto-pruebas> uv sync
Using CPython 3.13.7 interpreter at: C:\Python313\python.exe
Creating virtual environment at: .venv
Resolved 22 packages in 43ms
    Built proyecto-pruebas @ file:///C:/Users/vique/source/repos/proyecto-prueb
  · Preparing packages... (10/22)
dill ----- 94.79 KiB/117.21 KiB
  · Preparing packages... (10/22)
dill ----- 94.79 KiB/117.21 KiB
  · Preparing packages... (10/22)
dill ----- 94.79 KiB/117.21 KiB
packaging ----- 91.55 KiB/126.91 KiB
  · Preparing packages... (10/22)
dill ----- 94.79 KiB/117.21 KiB
packaging ----- 91.55 KiB/126.91 KiB
  · Preparing packages... (10/22)
dill ----- 94.79 KiB/117.21 KiB
packaging ----- 91.55 KiB/126.91 KiB

```

Explicación:

Se hace la sincronización del proyecto con uv, el mismo se encarga de crear el entorno virtual y se instalan las dependencias necesarias para el proyecto.

Verificación del ambiente

Versión de Python

Captura:

```

===== I passed in 0.18s =====
(proyecto-pruebas) PS C:\Users\vique\source\repos\proyecto-pruebas> python --version
Python 3.13.7

```

Explicación:

Se usa python 3.13.7, definida en el archivo .python-version, que es la versión que se instala al ejecutar **uv sync** y que se usa para crear el ambiente virtual del proyecto.

Versión de pip/UV

Captura:

```

Python 3.13.7
(proyecto-pruebas) PS C:\Users\vique\source\repos\proyecto-pruebas> uv --version
uv 0.12.15 (d35f1f270 2026-09-15 x86_64-pc-windows-msvc)

```

Explicación:

Se usa uv 0.12.15, que era la ultima versión disponible al momento de la instalación.

Importación exitosa de las librerías

Captura:

```

PS C:\Users\vique\source\repos\proyecto-pruebas> uv --version
uv 0.12.15 (d35f1f270 2026-09-15 x86_64-pc-windows-msvc)
PS C:\Users\vique\source\repos\proyecto-pruebas> uv sync
Using CPython 3.13.7 interpreter at: C:\Python313\python.exe
Creating virtual environment at: .venv
Resolved 22 packages in 43ms
    Built proyecto-pruebas @ file:///C:/Users/vique/source/repos/proyecto-prueb
:: Preparing packages... (10/22)
dill ----- 94.79 KiB/117.21 KiB
:: Preparing packages... (10/22)
dill ----- 94.79 KiB/117.21 KiB
:: Preparing packages... (10/22)
dill ----- 94.79 KiB/117.21 KiB
packaging ----- 91.55 KiB/126.91 KiB
:: Preparing packages... (10/22)
dill ----- 94.79 KiB/117.21 KiB
packaging ----- 91.55 KiB/126.91 KiB
:: Preparing packages... (10/22)
dill ----- 94.79 KiB/117.21 KiB
packaging ----- 91.55 KiB/126.91 KiB

```

Explicación:

Se importan exitosamente las librerías necesarias para el proyecto, incluyendo scipy, scikit-learn y pytest.

Ejecución de Pytest

Captura:

```

(proyecto-pruebas) PS C:\Users\vique\source\repos\proyecto-pruebas> uv run pytest
    Built proyecto-pruebas @ file:///C:/Users/vique/source/repos/proyecto-pruebas
Uninstalled 1 package in 11ms
Installed 1 package in 70ms
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\vique\source\repos\proyecto-pruebas
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.15.1
collected 1 item

tests\test_suma.py . [100%]

===== 1 passed in 0.18s =====

```

Explicación:

Se ejecuta Pytest exitosamente, lo que indica que las pruebas unitarias están funcionando correctamente.

Ejecución de Pylint

Captura:

```

Fetches rows: [{"id": 1, "created_at": "2026-09-17T03:34:04.472129+00:00", "name": "JJ the og"}]
(projecto-pruebas) PS C:\Users\vique\source\repos\projecto-pruebas> uv run pylint src
***** Module proyecto_pruebas.main
src\projecto_pruebas\main.py:1:0: C0114: Missing module docstring (missing-module-docstring)
src\projecto_pruebas\main.py:18:0: C0116: Missing function or method docstring (missing-function-docstring)
src\projecto_pruebas\main.py:24:0: C0116: Missing function or method docstring (missing-function-docstring)
src\projecto_pruebas\main.py:32:11: W0718: Catching too general exception Exception (broad-exception-caught)
)
***** Module proyecto_pruebas
src\projecto_pruebas\__init__.py:1:0: C0114: Missing module docstring (missing-module-docstring)
src\projecto_pruebas\__init__.py:1:0: C0116: Missing function or method docstring (missing-function-docstring)

-----
Your code has been rated at 7.86/10

```

Explicación:

Se ejecuta Pylint exitosamente, aunque da warnings, por ahora que es de prueba de conexión, se deja así.

Conexión exitosa con Supabase**Captura:**

```

Fetches rows: [{"id": 1, "created_at": "2026-09-17T03:34:04.472129+00:00", "name": "JJ the og"}]
(projecto-pruebas) PS C:\Users\vique\source\repos\projecto-pruebas> & c:\Users\vique\source\repos\projecto-pruebas\.venv\Scripts\python.exe c:/Users/vique/source/repos/projecto-pruebas/src/projecto_pruebas/main.py
SciPy: 1.18.1
Scikit-learn: 1.9.0
Pytest: 9.1.1
Successfully connected to Supabase!
Fetches rows: [{"id": 1, "created_at": "2026-09-17T03:34:04.472129+00:00", "name": "JJ the og"}]

```

Explicación:

Se establece una conexión exitosa con Supabase, lo que indica que el entorno está correctamente configurado.

Ejecución de una operación de prueba sobre Supabase**Captura:**

```

Fetches rows: [{"id": 1, "created_at": "2026-09-17T03:34:04.472129+00:00", "name": "JJ the og"}]
(projecto-pruebas) PS C:\Users\vique\source\repos\projecto-pruebas> & c:\Users\vique\source\repos\projecto-pruebas\.venv\Scripts\python.exe c:/Users/vique/source/repos/projecto-pruebas/src/projecto_pruebas/main.py
SciPy: 1.18.1
Scikit-learn: 1.9.0
Pytest: 9.1.1
Successfully connected to Supabase!
Fetches rows: [{"id": 1, "created_at": "2026-09-17T03:34:04.472129+00:00", "name": "JJ the og"}]

```

Explicación:

Se realiza una operación de prueba (select) sobre la tabla "users_test" en Supabase, y se obtiene una respuesta exitosa, lo que confirma que la conexión y la configuración de Supabase son correctas.

Conclusiones

- Se aprende que antes de empezar a probar hay que tener bien claro qué se va a probar y cómo, porque si no se pierde mucho tiempo después. Armar el plan y el diseño nos ayudó a ordenar las ideas.
- Montar el ambiente de pruebas en un entorno nuevo y más eficiente cómo uv nos permitió tener un ambiente de pruebas más controlado y reproducible, lo que es muy importante para poder hacer pruebas confiables.

Referencias

- <https://www.youtube.com/watch?v=VKiR6xBtWIQ>
- <https://supabase.com/docs>